

PPL20210120

SCHEME

Define a pure function (i.e. without using procedures with side effects, such as `set!`) which takes a multi-level list, i.e. a list that may contain any level of lists, and converts it into a data structure where each list is converted into a vector.

E.g.

The result of `(multi-list->vector '(1 2 (3 4) (5 (6)) "hi" ((3 4))))`

should be: `'#(1 2 #(3 4) #(5 #(6)) "hi" #(#(3) 4))`

HASKELL

Consider the following data structure for general binary trees:

```
data Tree a = Empty | Branch (Tree a) a (Tree a) deriving (Show, Eq)
```

Using the State monad as seen in class:

1) Define a monadic map for Tree, called `mapTreeM`.

2) Use `mapTreeM` to define a function which takes a tree and returns a tree containing list of elements that are all the data found in the original tree in a depth-first visit.

E.g.

From the tree: `(Branch (Branch Empty 1 Empty) 2 (Branch (Branch Empty 3 Empty) 4 Empty))`

we obtain:

```
Branch (Branch Empty [1] Empty) [1,2] (Branch (Branch Empty [1,2,3] Empty) [1,2,3,4] Empty)
```

ERLANG

Define a function for a proxy used to avoid to send PIDs; the proxy must react to the following messages:

- `{remember, PID, Name}`: associate the value Name with PID.
- `{question, Name, Data}`: send a question message containing Data to the PID corresponding to the value Name (e.g. an atom), like in `PID ! {question, Data}`
- `{answer, Name, Data}`: send an answer message containing Data to the PID corresponding to the value Name (e.g. an atom), like in `PID ! {answer, Data}`

SOLUTIONS

1)

```
(define (multi-list->vector lst)
  (cond
    ((not (list? lst)) lst)
    ((null? (filter list? lst)) (apply vector lst))
    (else (apply vector (map multi-list->vector lst)))))
```

2)

```
mapTreeM :: Monad m => (t -> m a) -> Tree t -> m (Tree a)
mapTreeM f Empty = return Empty
mapTreeM f (Branch lhs v rhs) = do
    lhs' <- mapTreeM f lhs
    v1 <- f v
    rhs' <- mapTreeM f rhs
    return (Branch lhs' v1 rhs')
```

```
depth_tree t = let (State f) = mapTreeM
    (\v -> do cur <- getState
              putState $ cur ++ [v]
              getState)
    in snd $ f []
```

3)

```
proxy(Table) ->
  receive
    {question, Name, Data} ->
      #{Name := Id} = Table,
      Id ! {question, Data},
      proxy(Table);
    {answer, Name, Data} ->
      #{Name := Id} = Table,
      Id ! {answer, Data},
      proxy(Table);
    {remember, PID, Name} ->
      proxy(Table#{Name => PID})
  end.
```